

EDA_Assignment

May 6, 2026

1 EDA - 1 Title: Bike Details Dataset

1.0.1 The dataset provided focuses on used bikes, capturing various attributes to analyze the second-hand bike

1.0.2 market. This dataset helps understand factors influencing resale prices, usage patterns, and market trends for

1.0.3 used bikes.

1.1 Source:

The dataset is available on Kaggle, simulating data commonly observed in real-world online bike resale platforms.

1.1.1 Description:

The dataset comprises 1061 records of used bike listings. These records include details about the bike's brand and model, selling price, kilometers driven, ownership history, and other attributes. It is particularly useful for resale value prediction, market analysis, and understanding buyer preferences.

The dataset consists of 7 features, combining both numeric and categorical data, offering a comprehensive view of the second-hand bike market.

Title: Bike Details Dataset

- name: Brand and model name of the bike (Categorical, e.g., "Royal Enfield Classic 350", "Honda Dio")Z
- selling_price: Listed selling price in INR (Numeric, e.g., 175000, 45000)Z
- year: Manufacturing year of the bike (Numeric, e.g., 2019, 2015)Z
- seller_type: Seller category, either "Individual" or "Dealer" (Categorical)Z
- owner: Ownership history, e.g., "1st owner", "2nd owner" (Categorical)Z
- km_driven: Total kilometers driven (Numeric, in km, e.g., 12000, 23000)Z
- ex_showroom_price: Original showroom price in INR (Numeric, e.g., 148114.0, 89643.0, etc.,).

```
[4]: # Import Some Libraries for analysis of data
```

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
```

```
[5]: df = pd.read_csv('BIKE DETAILS.csv')
```

```
[6]: df
```

```
[6]:
```

	name	selling_price	year	seller_type	\
0	Royal Enfield Classic 350	175000	2019	Individual	
1	Honda Dio	45000	2017	Individual	
2	Royal Enfield Classic Gunmetal Grey	150000	2018	Individual	
3	Yamaha Fazer FI V 2.0 [2016-2018]	65000	2015	Individual	
4	Yamaha SZ [2013-2014]	20000	2011	Individual	
...	...	...	...	...	
1056	Activa 3g	17000	2010	Individual	
1057	Honda CB twister	16000	2012	Individual	
1058	Bajaj Discover 125	15000	2013	Individual	
1059	Honda CB Shine	12000	2009	Individual	
1060	Bajaj Pulsar 150	10000	2008	Individual	

	owner	km_driven	ex_showroom_price
0	1st owner	350	NaN
1	1st owner	5650	NaN
2	1st owner	12000	148114.0
3	1st owner	23000	89643.0
4	2nd owner	21000	NaN
...	...	...	...
1056	1st owner	500000	52000.0
1057	1st owner	33000	51000.0
1058	2nd owner	35000	57000.0
1059	1st owner	53000	58000.0
1060	1st owner	92233	75000.0

[1061 rows x 7 columns]

```
[7]: df.drop_duplicates(inplace = True)
```

```
[8]: df.tail()
```

```
[8]:
```

	name	selling_price	year	seller_type	owner	\
1056	Activa 3g	17000	2010	Individual	1st owner	
1057	Honda CB twister	16000	2012	Individual	1st owner	
1058	Bajaj Discover 125	15000	2013	Individual	2nd owner	

1059	Honda CB Shine	12000	2009	Individual	1st owner
1060	Bajaj Pulsar 150	10000	2008	Individual	1st owner

	km_driven	ex_showroom_price
1056	500000	52000.0
1057	33000	51000.0
1058	35000	57000.0
1059	53000	58000.0
1060	92233	75000.0

```
[9]: df.info()
```

```
<class 'pandas.DataFrame'>
Index: 1055 entries, 0 to 1060
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   name                   1055 non-null   str
1   selling_price          1055 non-null   int64
2   year                   1055 non-null   int64
3   seller_type            1055 non-null   str
4   owner                  1055 non-null   str
5   km_driven              1055 non-null   int64
6   ex_showroom_price      622 non-null    float64
dtypes: float64(1), int64(3), str(3)
memory usage: 65.9 KB
```

```
[10]: df.describe()
```

```
[10]:
```

	selling_price	year	km_driven	ex_showroom_price
count	1055.000000	1055.000000	1055.000000	6.220000e+02
mean	59713.818957	2013.871090	34295.628436	8.806079e+04
std	56414.384170	4.305734	51708.286608	7.772528e+04
min	5000.000000	1988.000000	350.000000	3.049000e+04
25%	28000.000000	2011.000000	13500.000000	5.485200e+04
50%	45000.000000	2015.000000	25000.000000	7.275250e+04
75%	70000.000000	2017.000000	43000.000000	8.703150e+04
max	760000.000000	2020.000000	880000.000000	1.278000e+06

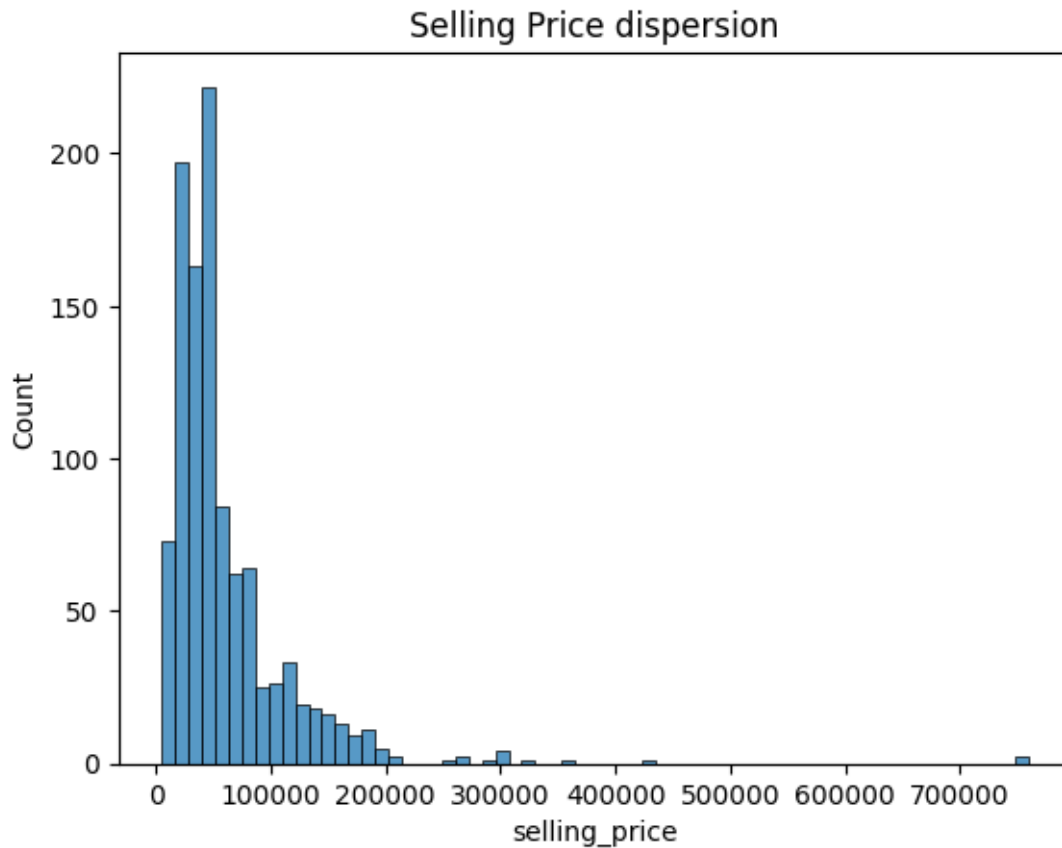
```
[11]: # What is the range of selling prices in the dataset?
df['Range'] = df.selling_price.max() - df.selling_price.min()
```

```
[12]: df.selling_price.max()
```

```
[12]: np.int64(760000)
```

2 What is the range of selling prices in the dataset?

```
[13]: plt.figure()
sns.histplot(data= df, x= 'selling_price')
plt.title('Selling Price dispersion')
plt.show()
```



2.0.1 we are see their are some outlier of price, but most of the bikes price are belong to 5000 to 200000 range.

```
[14]: q1, q2, q3 = df.selling_price.quantile([0.25, .5, .75])
```

```
[15]: df['range'] = q3 -q1
```

```
[16]: df
```

```
[16]:
```

	name	selling_price	year	seller_type	\
0	Royal Enfield Classic 350	175000	2019	Individual	
1	Honda Dio	45000	2017	Individual	
2	Royal Enfield Classic Gunmetal Grey	150000	2018	Individual	

3	Yamaha Fazer FI V 2.0 [2016-2018]	65000	2015	Individual
4	Yamaha SZ [2013-2014]	20000	2011	Individual
...	...	...	...	...
1056	Aktiva 3g	17000	2010	Individual
1057	Honda CB twister	16000	2012	Individual
1058	Bajaj Discover 125	15000	2013	Individual
1059	Honda CB Shine	12000	2009	Individual
1060	Bajaj Pulsar 150	10000	2008	Individual

	owner	km_driven	ex_showroom_price	Range	range
0	1st owner	350	NaN	755000	42000.0
1	1st owner	5650	NaN	755000	42000.0
2	1st owner	12000	148114.0	755000	42000.0
3	1st owner	23000	89643.0	755000	42000.0
4	2nd owner	21000	NaN	755000	42000.0
...	...	...	...	...	...
1056	1st owner	500000	52000.0	755000	42000.0
1057	1st owner	33000	51000.0	755000	42000.0
1058	2nd owner	35000	57000.0	755000	42000.0
1059	1st owner	53000	58000.0	755000	42000.0
1060	1st owner	92233	75000.0	755000	42000.0

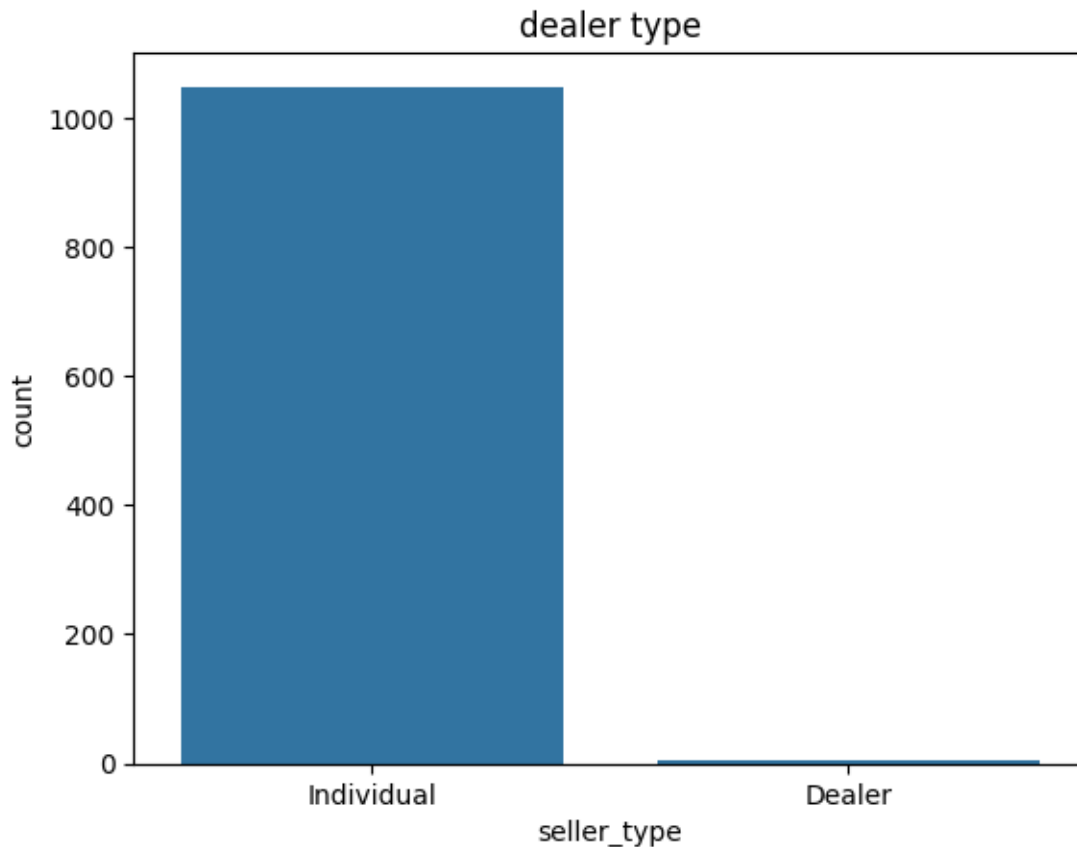
[1055 rows x 9 columns]

3 What is the median selling price for bikes in the dataset?

```
[20]: meadian_Bike = df.selling_price.median()
      meadian_Bike
```

```
[20]: np.float64(45000.0)
```

```
[29]: # What is the most common seller type
      df.seller_type.unique()
      sns.countplot(df, x= 'seller_type', )
      plt.title('dealer type')
      plt.show()
```



3.0.1 in that figure see the most common dealer is individual

4 4. How many bikes have driven more than 50,000 kilometers?

```
[40]: count = df[df['km_driven'] > 50000].shape[0]
      print('driven more than 50,000 kilometers bikes-', count)
```

driven more than 50,000 kilometers bikes- 168

[33]:

```
[33]:
```

	name	selling_price	year	seller_type
0	Royal Enfield Classic 350	175000	2019	Individual
1	Honda Dio	45000	2017	Individual
2	Royal Enfield Classic Gunmetal Grey	150000	2018	Individual
3	Yamaha Fazer FI V 2.0 [2016-2018]	65000	2015	Individual
4	Yamaha SZ [2013-2014]	20000	2011	Individual
...	...	...	...	...
1056	Activa 3g	17000	2010	Individual
1057	Honda CB twister	16000	2012	Individual

1058	Bajaj Discover 125	15000	2013	Individual
1059	Honda CB Shine	12000	2009	Individual
1060	Bajaj Pulsar 150	10000	2008	Individual

	owner	km_driven	ex_showroom_price
0	1st owner	350	NaN
1	1st owner	5650	NaN
2	1st owner	12000	148114.0
3	1st owner	23000	89643.0
4	2nd owner	21000	NaN
...	...	...	...
1056	1st owner	500000	52000.0
1057	1st owner	33000	51000.0
1058	2nd owner	35000	57000.0
1059	1st owner	53000	58000.0
1060	1st owner	92233	75000.0

[1055 rows x 4 columns]

5. What is the average km_driven value for each ownership type?

```
[51]: df1 = df[['owner', 'km_driven']]
      df1.groupby('owner').mean()
```

```
[51]:          km_driven
owner
1st owner    32732.710240
2nd owner    39288.991870
3rd owner    33292.181818
4th owner    311500.000000
```

6. What proportion of bikes are from the year 2015 or older

```
[54]: # Count bikes from 2015 or older
      older_bikes = df[df['year'] <= 2015].shape[0]

      # Total bikes
      total_bikes = df.shape[0]

      # Proportion
      proportion = older_bikes / total_bikes

      print(proportion * 100)
```

56.58767772511848

7 7. What is the trend of missing values across the datasetD

```
[60]: missing_counts = df.isnull().sum()
missing_percent = (df.isnull().sum() / len(df)) * 100

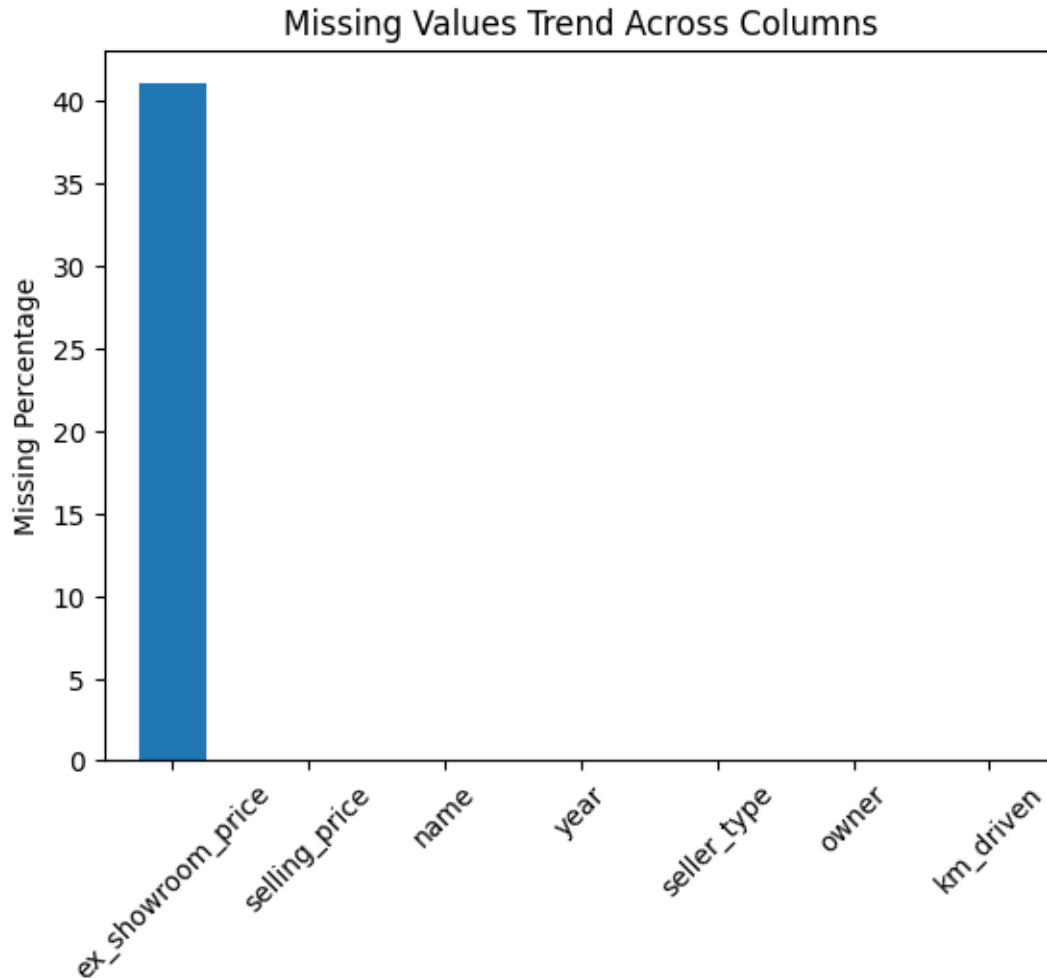
missing_df = pd.DataFrame({
    'Missing_Count': missing_counts,
    'Missing %': missing_percent
}).sort_values(by='Missing %', ascending=False)

print(missing_df)
```

	Missing_Count	Missing %
ex_showroom_price	433	41.042654
selling_price	0	0.000000
name	0	0.000000
year	0	0.000000
seller_type	0	0.000000
owner	0	0.000000
km_driven	0	0.000000

```
[62]: import matplotlib.pyplot as plt

missing_percent.sort_values(ascending=False).plot(kind='bar')
plt.ylabel('Missing Percentage')
plt.title('Missing Values Trend Across Columns')
plt.xticks(rotation=45)
plt.show()
```

8 8. What is the highest ex_showroom_price recorded, and for which bike

```
[83]: max_price = df['ex_showroom_price'].max()

df[df['ex_showroom_price'] == max_price][['name', 'ex_showroom_price']]
```

```
[83]:
```

	name	ex_showroom_price
134	Harley-Davidson Street Bob	1278000.0

9 9 What is the total number of bikes listed by each seller type

```
[91]: df['seller_type'].value_counts().reset_index()
```

```
[91]: seller_type  count
      0  Individual  1049
      1     Dealer     6
```

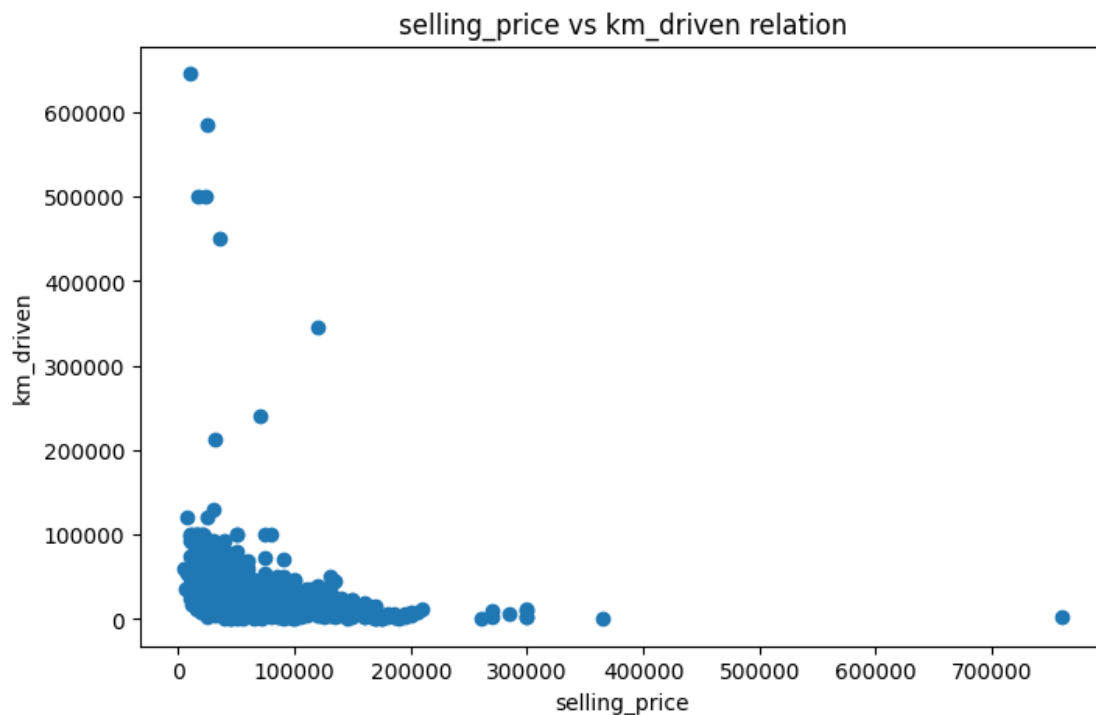
10 10. What is the relationship between selling_price and km_driven for first-owner bikesD

```
[93]: first_owner = df[df['owner'] == '1st owner']

first_owner[['selling_price', 'km_driven']].corr()
```

```
[93]:          selling_price  km_driven
selling_price      1.000000 -0.242205
km_driven          -0.242205  1.000000
```

```
[97]: plt.figure(figsize=(8,5))
plt.scatter(first_owner['selling_price'],first_owner['km_driven'] )
plt.xlabel('selling_price')
plt.ylabel('km_driven')
plt.title('selling_price vs km_driven relation')
plt.show()
```



10.0.1 As km_driven increases, selling_price decreases.

11. Identify and remove outliers in the km_driven column using the IQR method

```
[99]: Q1 = df['km_driven'].quantile(0.25)
      Q3 = df['km_driven'].quantile(0.75)

      IQR = Q3 - Q1
      lower_limit = Q1 - 1.5 * IQR
      upper_limit = Q3 + 1.5 * IQR

      outliers = df[(df['km_driven'] < lower_limit) |
                    (df['km_driven'] > upper_limit)]

      print(outliers)
```

	name	selling_price	year	seller_type	\
91	Honda Activa [2000-2015]	22989	2012	Individual	
93	Suzuki GS150R	30000	2010	Individual	
115	Hero Honda Passion Plus	7500	2004	Individual	
135	Hero Honda CBZ extreme	20000	2007	Individual	
154	Hero Xtreme Sports	22000	2014	Individual	
184	Honda CB Unicorn 160	75000	2015	Individual	
363	Hero Honda CD100SS	10000	1997	Individual	
470	Honda Activa [2000-2015]	23000	2009	Individual	
474	Hero Honda Passion PRO [2012]	12000	2012	Individual	
477	Honda Activa [2000-2015]	25000	2009	Individual	
511	Hero Splendor Plus	12000	2007	Individual	
535	Hero Honda CD Deluxe	12000	2007	Individual	
548	Bajaj Xcd	25000	2007	Individual	
575	Hero Honda Karizma	40000	2011	Individual	
592	TVS Star City	21000	2011	Individual	
598	Yamaha Fazer [2009-2016]	25000	2013	Individual	
610	Hero Honda Splendor Plus	15000	2006	Individual	
630	Hero Honda Splendor Plus	10000	2007	Individual	
642	TVS Apache RTR 160	15000	2009	Individual	
668	Yamaha FZ16	40000	2016	Individual	
687	Honda Activa [2000-2015]	16600	2010	Individual	
695	Yamaha Fazer [2009-2016]	50000	2012	Individual	
712	Bajaj Xcd	15000	2008	Individual	
714	Honda CB Hornet 160R	70000	2017	Individual	
716	TVS Streak	10000	2009	Individual	
726	Honda CB Hornet 160R	80000	2016	Individual	
782	Bajaj Pulsar 150 [2001-2011]	12000	2007	Individual	
797	Royal Enfield Electra Twinspark	120000	2016	Individual	
830	Bajaj Pulsar 150	21000	2007	Individual	
872	Honda Shine	30000	2013	Individual	

877	Hero Glamour 125	35000	2014	Individual
901	Hero Honda Passion Plus	25000	2008	Dealer
903	Hero Honda Splendor Plus	20000	2004	Individual
908	Hero Super Splendor	30000	2011	Individual
930	Hero Maestro	50000	2016	Individual
1039	Honda Karizma	31000	2012	Individual
1056	Activa 3g	17000	2010	Individual
1060	Bajaj Pulsar 150	10000	2008	Individual

	owner	km_driven	ex_showroom_price
91	1st owner	500000	50267.0
93	1st owner	130000	70851.0
115	1st owner	120000	NaN
135	1st owner	90000	NaN
154	1st owner	99000	78350.0
184	1st owner	100000	79233.0
363	1st owner	646000	NaN
470	1st owner	95000	50267.0
474	1st owner	90000	NaN
477	1st owner	585659	50267.0
511	1st owner	100000	NaN
535	2nd owner	100000	NaN
548	2nd owner	100000	NaN
575	2nd owner	340000	NaN
592	1st owner	101000	47108.0
598	1st owner	120000	84751.0
610	1st owner	100000	NaN
630	1st owner	98000	NaN
642	4th owner	880000	NaN
668	1st owner	93000	78712.0
687	1st owner	100000	50267.0
695	1st owner	100000	84751.0
712	1st owner	100000	NaN
714	1st owner	240000	92210.0
716	1st owner	100000	NaN
726	1st owner	100000	92661.0
782	2nd owner	100000	NaN
797	1st owner	345000	NaN
830	1st owner	100000	NaN
872	1st owner	93000	NaN
877	1st owner	450000	61600.0
901	2nd owner	100000	NaN
903	2nd owner	127000	NaN
908	1st owner	89000	NaN
930	1st owner	100000	49412.0
1039	1st owner	213000	105000.0
1056	1st owner	500000	52000.0
1060	1st owner	92233	75000.0

```
[100]: # remove the outlier
df_clean = df[(df['km_driven'] >= lower_limit) &
              (df['km_driven'] <= upper_limit)]

print(df_clean.shape)
```

(1017, 7)

```
[101]: df_clean
```

```
[101]:
```

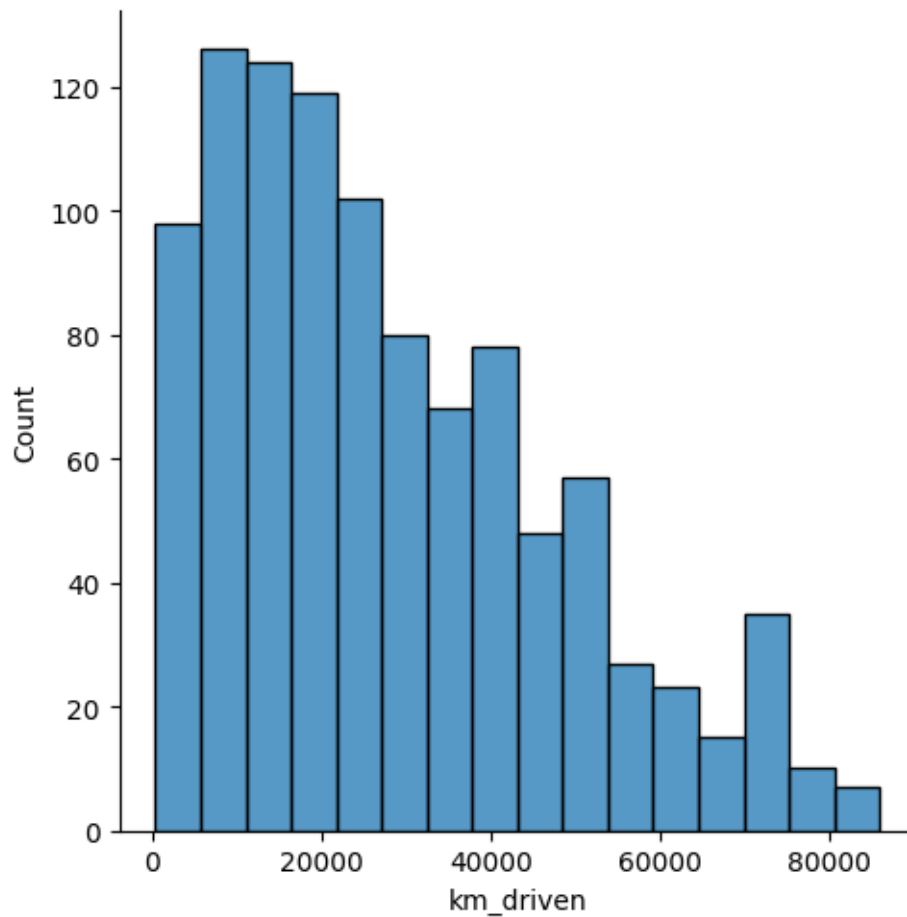
	name	selling_price	year	seller_type	\
0	Royal Enfield Classic 350	175000	2019	Individual	
1	Honda Dio	45000	2017	Individual	
2	Royal Enfield Classic Gunmetal Grey	150000	2018	Individual	
3	Yamaha Fazer FI V 2.0 [2016-2018]	65000	2015	Individual	
4	Yamaha SZ [2013-2014]	20000	2011	Individual	
...	...	...	...	...	
1054	Hero CBZ Xtreme	20000	2010	Individual	
1055	Bajaj ct 100	18000	2017	Individual	
1057	Honda CB twister	16000	2012	Individual	
1058	Bajaj Discover 125	15000	2013	Individual	
1059	Honda CB Shine	12000	2009	Individual	

	owner	km_driven	ex_showroom_price
0	1st owner	350	NaN
1	1st owner	5650	NaN
2	1st owner	12000	148114.0
3	1st owner	23000	89643.0
4	2nd owner	21000	NaN
...	...	...	...
1054	1st owner	50000	78700.0
1055	1st owner	35000	32000.0
1057	1st owner	33000	51000.0
1058	2nd owner	35000	57000.0
1059	1st owner	53000	58000.0

[1017 rows x 7 columns]

```
[109]: sns.displot(df_clean['km_driven'] )
```

```
[109]: <seaborn.axisgrid.FacetGrid at 0x25d5d52a490>
```



12 12 Perform a bivariate analysis to visualize the relationship between year and selling_priceB

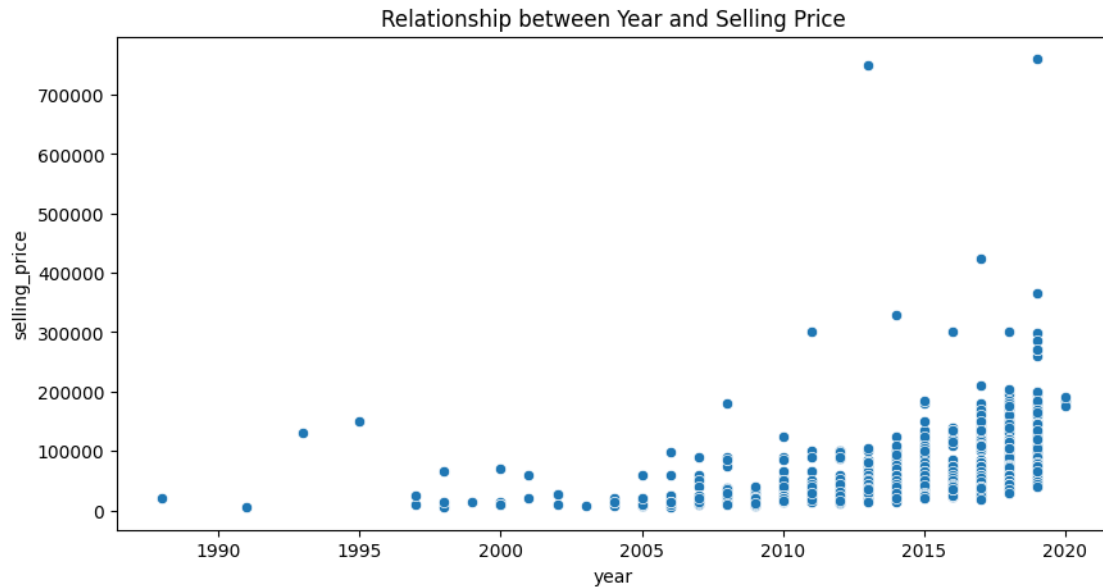
```
[110]: import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))

sns.scatterplot(x='year',
                y='selling_price',
                data=df)

plt.title('Relationship between Year and Selling Price')

plt.show()
```



13 13 What is the average depreciation in selling price based on the bike's age current year - manufacturing year

```
[112]: df['age'] = 2026 - df['year']
depreciation = df.groupby('age')['selling_price'].mean().reset_index()

print(depreciation)
```

	age	selling_price
0	6	183333.333333
1	7	119689.511628
2	8	87603.915385
3	9	78962.121212
4	10	58469.018692
5	11	56500.000000
6	12	48668.131868
7	13	51152.777778
8	14	35976.637681
9	15	35655.721311
10	16	31993.220339
11	17	22240.740741
12	18	34289.285714
13	19	24927.586207
14	20	23380.000000
15	21	16978.571429
16	22	15100.000000
17	23	8000.000000

```

18  24  20666.666667
19  25  40000.000000
20  26  20833.333333
21  27  15000.000000
22  28  28333.333333
23  29  17500.000000
24  31  150000.000000
25  33  130000.000000
26  35   6000.000000
27  38  20000.000000

```

```

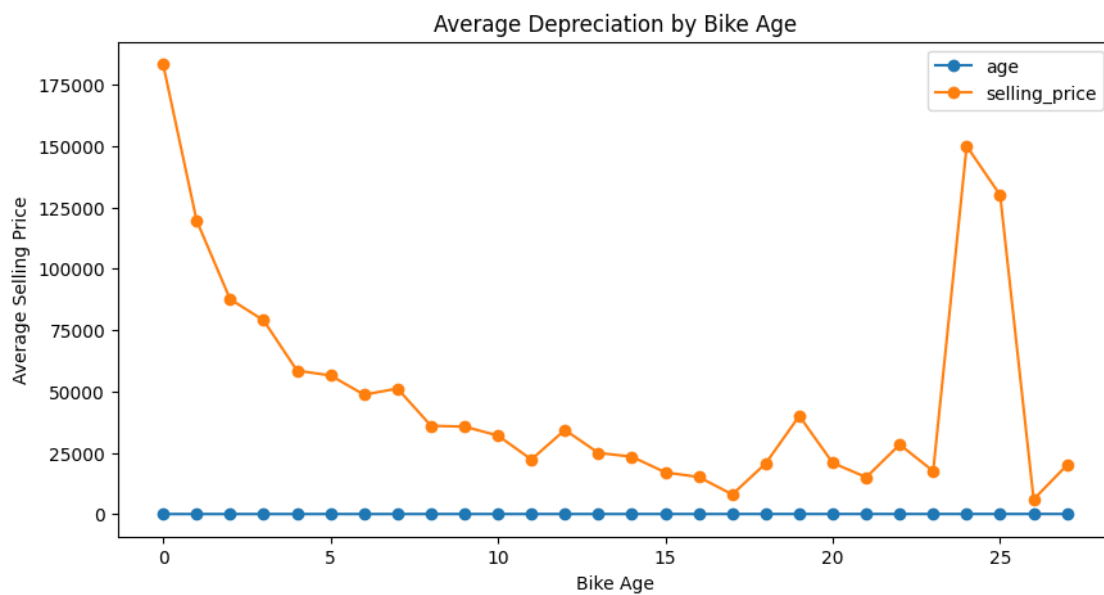
[114]: import matplotlib.pyplot as plt

depreciation.plot(kind='line',
                  marker='o',
                  figsize=(10,5))

plt.xlabel('Bike Age')
plt.ylabel('Average Selling Price')
plt.title('Average Depreciation by Bike Age')

plt.show()

```

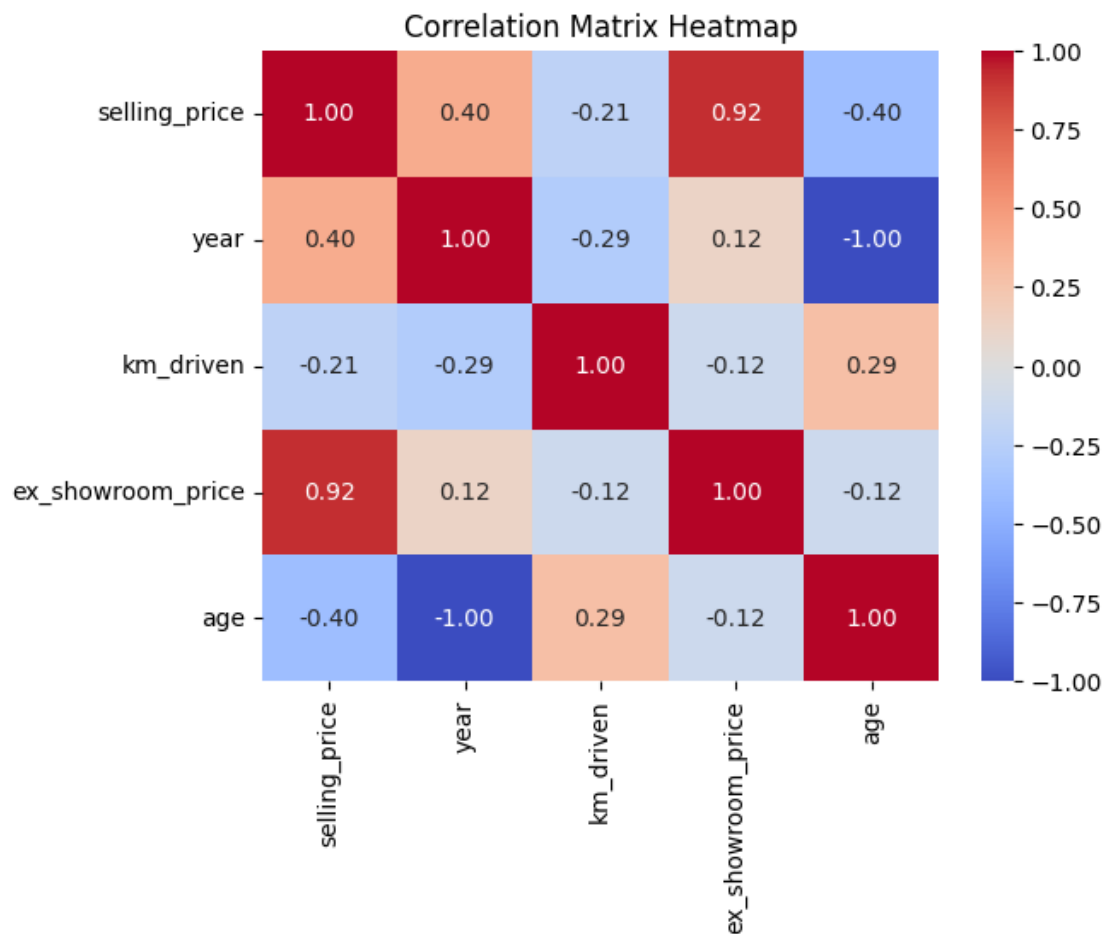


14 15 Develop a correlation matrix for numeric columns and visualize it using a heatmap.

```
[120]: numeric_df = df.select_dtypes(include=['int64', 'float64'])
corr_matrix = numeric_df.corr()

plt.figure()
sns.heatmap(corr_matrix,
            annot=True,
            cmap='coolwarm',
            fmt='.2f')
plt.title('Correlation Matrix Heatmap')

plt.show()
```



```
[ ]:
```